



Practical 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Algorithms:

Algorithm
merge sort(A, low, high)

1. if low < high
2. mid = (low + high) / 2
3. merge sort(A, low, mid)
4. merge sort(A, mid + 1, high)
5. merge(A, low, mid + 1, high)
6. end if

merge(A, low, mid + 1, high)

1. create a temporary array
2. i = low
3. j = mid + 1
4. k = low
5. while i <= mid and j <= high
6. if A[i] < A[j]
temp[k] = A[i]
i++
k++
7. else
temp[k] = A[j]
j++
k++
8. end while

end while

selection sort

Algorithm :

1. start
2. for i = 0 to n - 2
3. min = i
4. for j = i + 1 to n - 1
5. if A[j] < A[min]
6. min = j
7. end if
8. swap A[i] and A[min]
9. end for
10. stop

Time complexity
inner loop i = 0 to n - 1

Practical 1
bubble sort

Algorithm

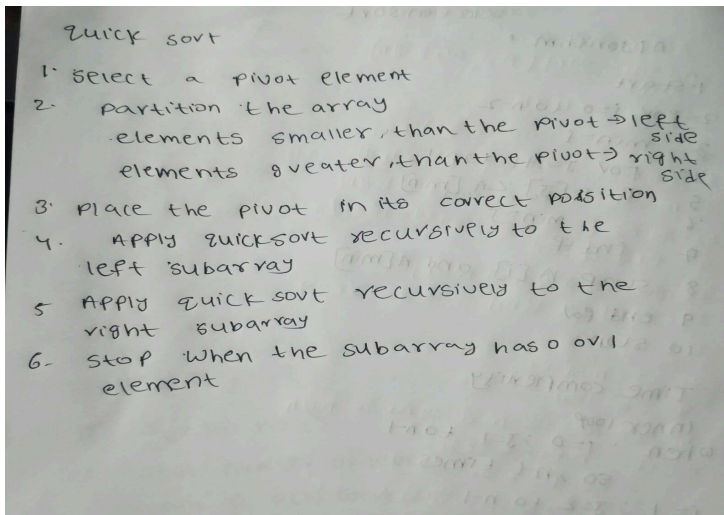
1. start
2. Repeat for i = 0 to n - 2
3. Repeat for j = 0 to n - i - 1
4. compare A[j] and A[j + 1]
5. if A[j] > A[j + 1]
swap A[j] and A[j + 1]
6. end if
7. end for
8. end for
9. stop

complexity:

Insertion sort

Algorithm

1. start
2. for i = 1 to n - 1
3. key = A[i]
4. j = i - 1
5. while j > 0 and A[j] > key
6. A[j + 1] = A[j]
7. j = j - 1
8. end while
9. A[j + 1] = key
10. end for
11. stop



CODE :

```
#include <iostream>
using namespace std;

// BUBBLE SORT
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
    }  
}  
  
// SELECTION SORT  
void selectionSort(int arr[], int n)  
{  
    for (int i = 0; i < n - 1; i++)  
    {  
        int minIndex = i;  
  
        for (int j = i + 1; j < n; j++)  
        {  
            if (arr[j] < arr[minIndex])  
            {  
                minIndex = j;  
            }  
        }  
  
        int temp = arr[i];  
        arr[i] = arr[minIndex];  
        arr[minIndex] = temp;  
    }  
}  
  
// INSERTION SORT  
void insertionSort(int arr[], int n)  
{  
    for (int i = 1; i < n; i++)  
    {  
        int key = arr[i];  
        int j = i - 1;  
  
        while (j >= 0 && arr[j] > key)  
        {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
    }  
}
```



```
        arr[j + 1] = key;
    }
}

// MERGE SORT
void merge(int arr[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = 0;

    int size = high - low + 1;
    int* temp = new int[size];

    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }
}
```

```
while (j <= high)
{
    temp[k] = arr[j];
    j++;
    k++;
}

for (i = low, k = 0; i <= high; i++, k++)
{
    arr[i] = temp[k];
}

delete[] temp;
}

void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = low + (high - low) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}

// QUICK SORT
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++)
```

```
{
    if (arr[j] < pivot)
    {
        i++;

        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

// Put pivot in correct position
int temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;

int pivotIndex = i + 1;

quickSort(arr, low, pivotIndex - 1);
quickSort(arr, pivotIndex + 1, high);
}
}

// ===== MAIN FUNCTION =====
int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int* arr = new int[n];

    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
}
```

```
}

int choice;

cout << "\n--- SORTING MENU--- \n";
cout << "1. Bubble Sort\n";
cout << "2. Selection Sort\n";
cout << "3. Insertion Sort\n";
cout << "4. Merge Sort\n";
cout << "5. Quick Sort\n";
cout << "Enter your choice: ";
cin >> choice;

switch (choice)
{
    case 1:
        bubbleSort(arr, n);
        cout << "\nBubble Sort selected.\n";
        break;

    case 2:
        selectionSort(arr, n);
        cout << "\nSelection Sort selected.\n";
        break;

    case 3:
        insertionSort(arr, n);
        cout << "\nInsertion Sort selected.\n";
        break;

    case 4:
        mergeSort(arr, 0, n - 1);
        cout << "\nMerge Sort selected.\n";
        break;

    case 5:
        quickSort(arr, 0, n - 1);
```

```
        cout << "\nQuick Sort selected.\n";
        break;

    default:
        cout << "\nInvalid choice!";
        delete[] arr;
        return 0;
    }

    cout << "Sorted array: ";

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    cout << endl;

    delete[] arr;

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 4
Enter 4 elements: 4
5
1
5

===== SORTING MENU =====
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Merge Sort
5. Quick Sort
Enter your choice: 1

Bubble Sort selected.
Sorted array: 1 4 5 5
```




Selection Sort selected.

Sorted array: 1 4 5 5

Insertion Sort selected.

Sorted array: 1 4 5 5

Merge Sort selected.

Sorted array: 1 4 5 5

Quick Sort selected.

Sorted array: 1 4 5 5

TIME COMPLEXITY:

q. Stop bubble sort
Time complexity:
 $i=0; j=0$ to $n-2$
 $\rightarrow n-1$ times
 $i=1; j=0$ to $n-2$
 $i=2; j=0$ to $n-3$
so $(n-1) + (n-2) + (n-3) + \dots + 1$
 $\Rightarrow (n-1) \frac{(n-1+1)}{2} = T(n)$
 $T(n) = \frac{(n^2 - n)}{2}$
we ignore constant and take high
 $T(n) = n^2$
so, $T(n) = O(n^2)$
for all best case, Average case, worst case
are same Time complexity

Stop Insertion sort
Time complexity
Average case
outer for loop
 $1, 2, 3, \dots, n-1$
so it returns
 $n-1$ times
inner while loop
 $i=1; j=0 \rightarrow 1$ times
 $i=2; j=1, 0 \rightarrow 2$ times
 $i=3; j=2, 1, 0 \rightarrow 3$ times
 $\therefore 1 + 2 + 3 + \dots + (n-1)$
so $(n-1) \frac{(n-1+1)}{2}$
 $\Rightarrow T(n) = \frac{(n^2 - n)}{2}$
 $= n^2/2 - n/2$
 $T(n) = O(n^2)$
Average case, and worst case are same.
Time complexity.



q. end for
10. stop

selection sort

Time complexity

Inner loop for all best case, Average case, worst case all give same

when $i=0$; $j=1$ to $n-1$
so, $n-1$ times

$i=1$; $j=2$ to $n-1$
so, $n-2$ times

$i=2$; $j=3$ to $n-1$
so, $n-3$ times

now $(n-1) + (n-2) + (n-3) + \dots + 1$

$n(n-1)/2 \Rightarrow (n^2-n)/2$

so, $T(n) = O(n^2)$

Time complexity: mergesort

Total no. of call & work done each call

$n=4$

$R(4) \rightarrow 2^0$ calls

$F(2) \rightarrow 2^1$ calls

$F(1) \rightarrow 2^2$ calls

$F(1) \rightarrow 2^2$ calls

$F(1) \rightarrow 2^2$ calls

$F(1) \rightarrow 2^2$ calls

so $\frac{n}{2^k} = 1 \Rightarrow n = 2^k$

so total number of levels $= \log_2 n = k$

in 1st call $\Rightarrow O(n)$ work done

in 2nd call $\Rightarrow O(n)$ work done $(\frac{n}{2} + \frac{n}{2}) = n$

for each level $O(n)$ work done

$\log_2 n \times O(n)$

for all best case, Average case, worst case

$T(n) = O(n \log n)$

element

quicksort

Time complexity

1. Partition loop

for $(int j = low; j < high; j++)$

run approximately times

$O(n)$

2. Best case

suppose the pivot divides the array equally

n

$n/2$ $n/2$

At each level we do $O(n)$ work

no. of levels $\Rightarrow \log_2 n$

Best case $= O(n) + O(\log n)$

$= O(n \log n)$

Average case

normally, the pivot produces balanced partition

Average case $= O(n \log n)$

3. Worst case

the pivot is always the smallest or largest element

$n + n-1 + n-2 + n-3 + \dots + 1$

$\therefore n(n+1)/2$

$\Rightarrow O(n^2)$